

Vehicle-mounted balance ball motion control system

Vehicle-mounted balance ball motion control system

- 1.About hardware
- 2.Software Environment
- 3.Structural Description
 - 3.1 Component List and Material Preparation
 - 3.2 Actual appearance drawing
 - 3.3 Assembly Instructions
- 4. Brief Principle
 - 4.1 Wiring Pins
 - 4.2 Physical Wiring Diagram
 - 4.3 Control Principle
- 5.Project Document Structure

- 6. Core Parameter Configuration
 - 6.1 Servo Controller PID (app_k230.c)
 - 6.2 Tracking PID (app_irtracking.c)
 - 6.3 Line-following speed for each task (questions.c)
 - 6.4 Task timeout settings (questions.c)
- 7. Task Description
 - 7.1 State Machine Structure (questions.h)
 - 7.2 Task Details
- 8. Key Code Explanation
 - 8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)
 - 8.2 Line tracking — LineWalking() (app_irtracking.c)
 - 8.3 Line tracking + ball control combination — LineWalking_WithBall() (app_irtracking.c)
 - 8.4 Task 2: Steel ball reciprocating (questions.c)
- 9. Operation Procedure

1.About hardware

Components	Model/Description
Main control	MSPM0G3507 board
Camera	K230 AI Vision Module (YOLOv8 Steel Ball Detection)
Line tracking	8-channel Line tracking (I2C 0x12)
Display	OLED 128×32 (I2C)
Servo	9G servo (ball bearing tilt control)
Motor	Two-wheeled L-type 520 encoder motor (serial port driver board)

Components	Model/Description
Button	K1 expansion board single-button control

2. Software Environment

- **IDE:** Keil MDK V5 (ARM Compiler V6.7)
- **SDK:** TI MSPM0 SDK 2.02.00.05

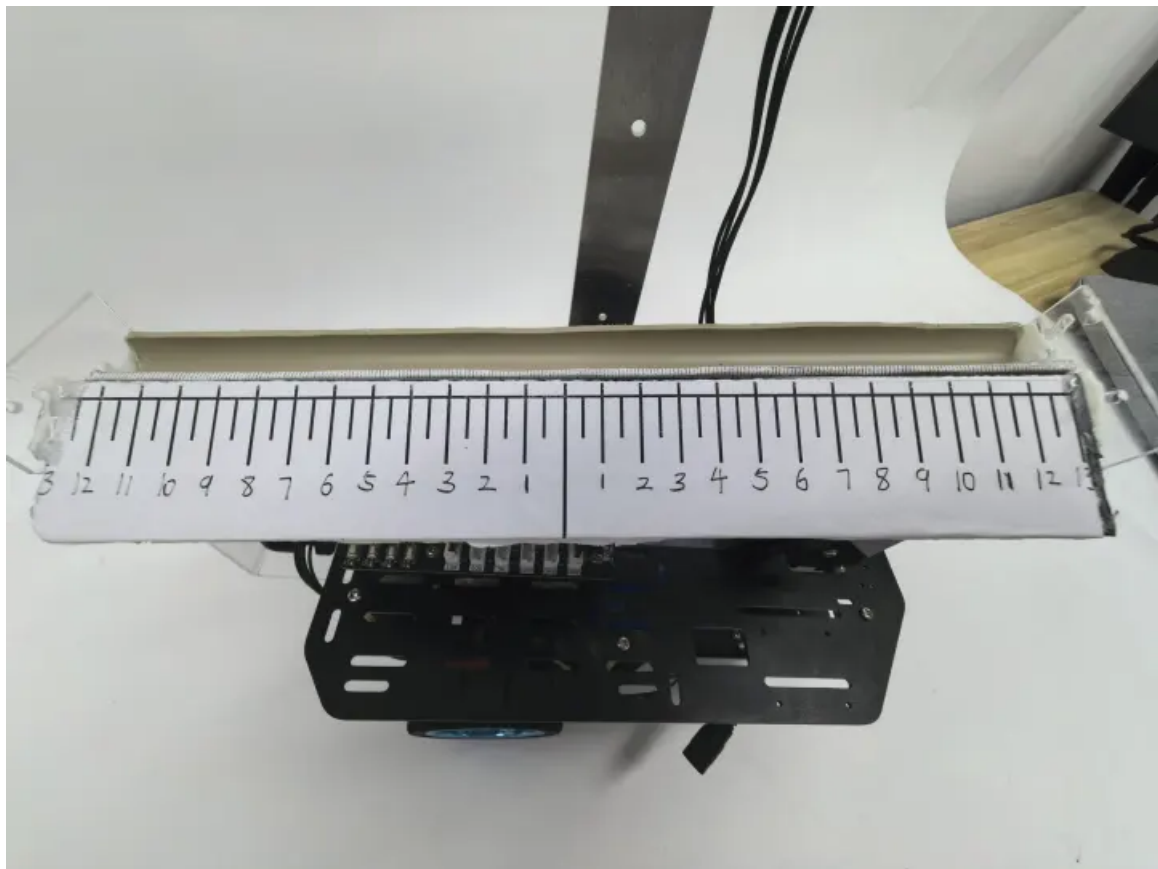
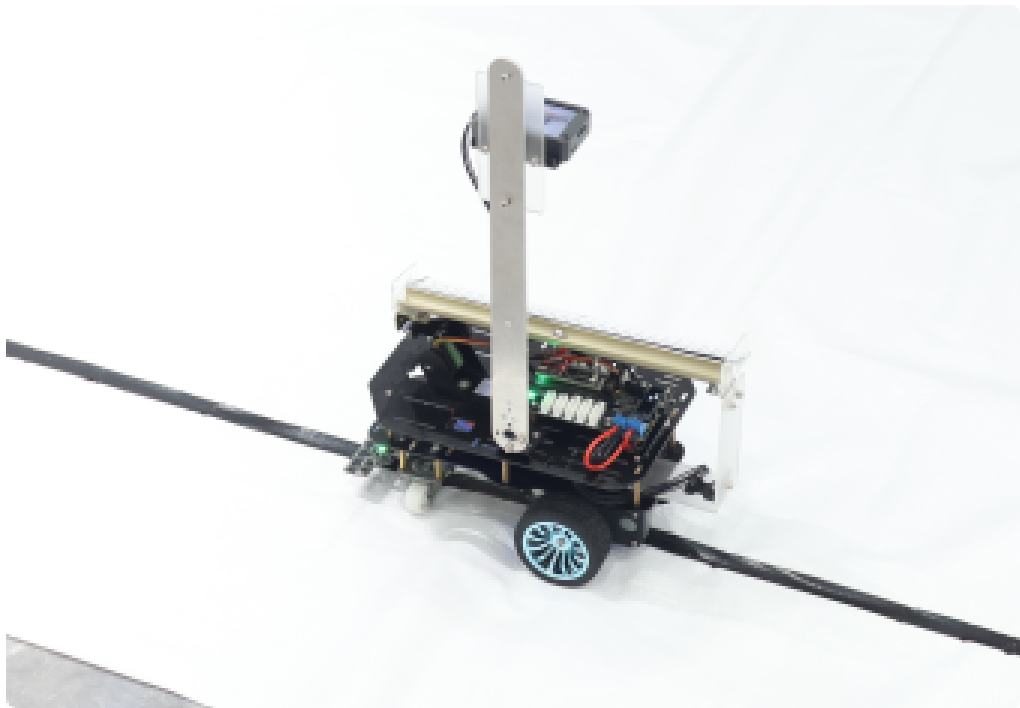
3. Structural Description

3.1 Component List and Material Preparation

- **Chassis Section:** Black etched metal base plate, drive motor, wheels. The chassis has pre-drilled screw holes and wiring holes.
- **Robotic Arm Structure:** Blue/black metal servo bracket, metal connecting rod, metal L-shaped/Straight connector, servo, metal flange.
- **Horizontal Ruler Slide:** Transparent acrylic shell, handwritten numeral ruler and zero reference line, internal transmission guide rail/lead screw.
- **Top Vision Bracket:** Brushed metal strip (aluminum alloy/stainless steel), transparent acrylic L-shaped bracket, K230 module.
- **Hardware Accessories:** M2/M3 combination screws, nuts, washers, black cable ties, heat shrink tubing.

Note: Besides the accessories included in the v2 chassis kit, the 9G servo, K230, and tracking module can be purchased from our store. Other connecting rods, rulers, water pipes, and other connecting hardware accessories need to be obtained independently.**

3.2 Actual appearance drawing

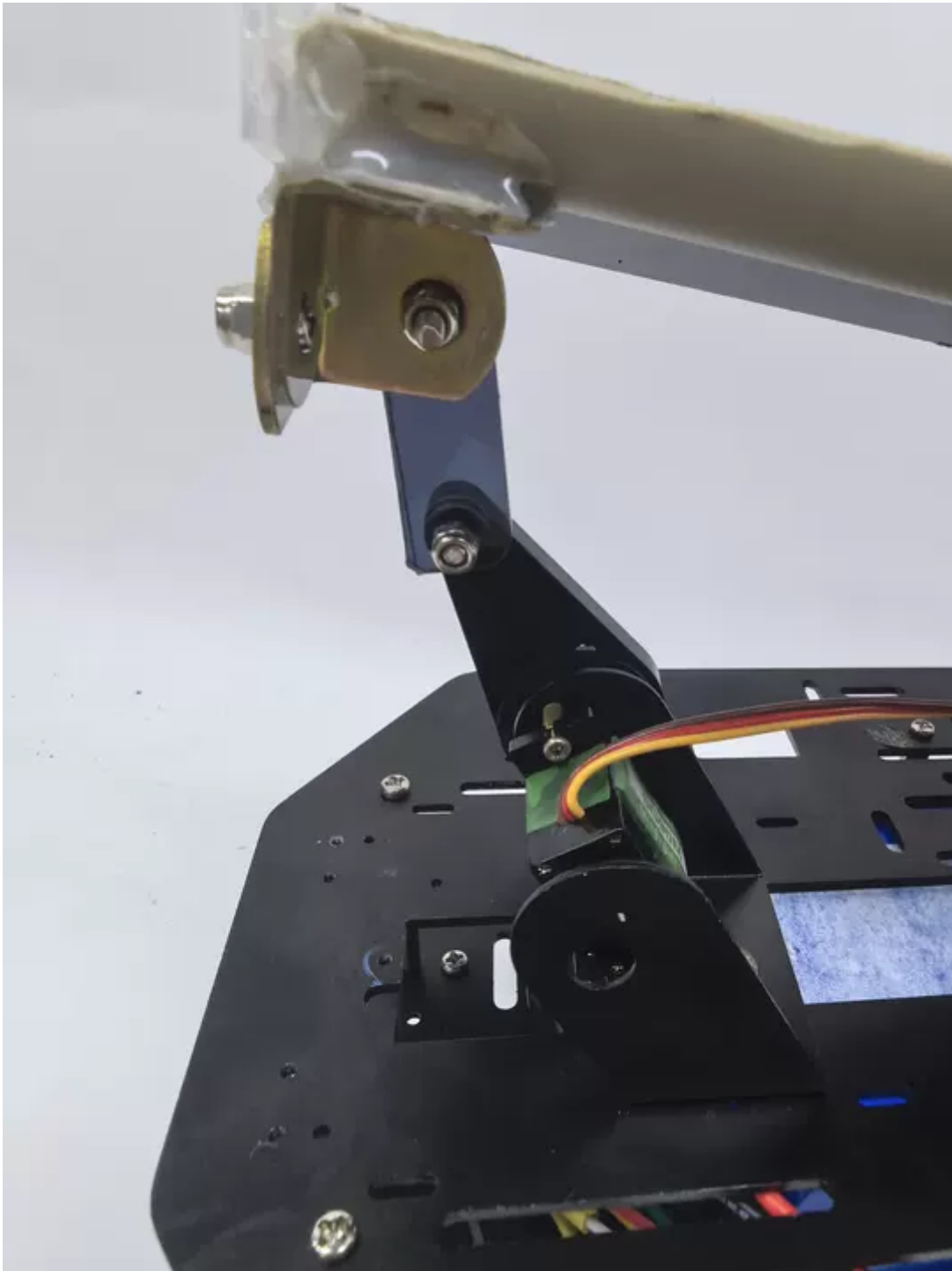












3.3 Assembly Instructions

Step 1: Install the Mobile Chassis and Motors

- Secure the two drive motors (with wheels) to both sides of the black metal base plate. Ensure the wheels are facing the same direction and in full contact with the ground.
- Install the main control board mspm0g3507 and secure it to the pre-drilled holes in the chassis using screws. Ensure the main control board avoids the rotation space of the wheels and motors.

Step 2: Build the Level Marker and Translation Slide

- *Refer to Figure 5:* Assemble the transparent acrylic shell and embed the cardboard with the graduated scale inside. This structure is fixed to the front of the chassis.
- Ensure the guide rails or lead screws inside the slide are level; this is a crucial reference point for subsequent robotic arm coordinate calibration.

Step 3: Assemble the Robotic Arm Joints

- *Refer to Figures 3 and 7:* Insert the green servo into the black metal bracket and tighten it with the matching screws.
- Install the servo arm and connect it to the blue/black metal linkage.
- Refer to Figure 7 to assemble the end effector (connector/claw). The image shows this component reinforced with a significant amount of **transparent acrylic and hot melt adhesive/resin**. In actual assembly, it is recommended to use 3D printed parts or metal parts to ensure sufficient structural strength.
- Secure the servo base to the rear of the chassis (refer to the layout in Figure 7).

Step 4: Install the Vertical Metal Bracket

- Refer to Figures 1, 4, and 6: Take a brushed metal strip and install it vertically into the mounting slot in the center or front side of the chassis.
- When using screws for securing, ensure the metal strip is perpendicular to the chassis plane and does not wobble. The nut in Figure 6 is tightened through the back.

Step 5: Install the Top Vision Sensor

- Refer to Figure 35: With the back of the K230 module facing outwards. Use a transparent acrylic L-shaped bracket and M3 screws to secure the module to the top of the vertical metal bracket.
- Adjust the sensor's orientation to ensure it can see the track and scale in front.

4. Brief Principle

4.1 Wiring Pins

Four-Channel Motor Driver Board:

Motor drive board	MSPM0G3507
RX2	PB6
TX2	PB7
GND	GND
5V	5V

Motor wiring:

Motor	Drive board
-------	-------------

Motor	Drive board
M2	M-
V	3V3
A	H1B
B	H1A
G	GND
M1	M+

8-channel tracking module:

8-channel tracking module	MSPM0G3507
5V	5V
SCL	PA15
SDA	PA16
GND	GND

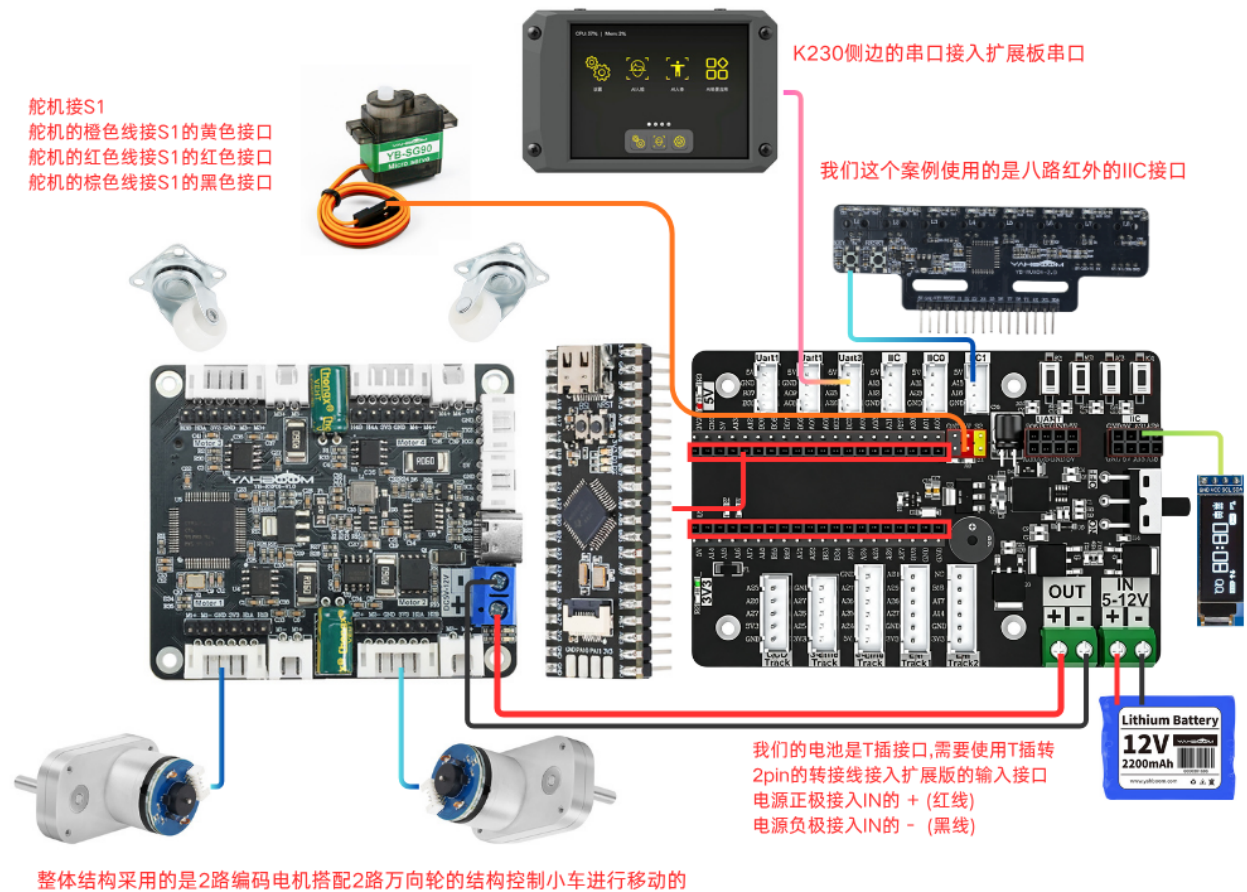
OLED:

OLED	MSPM0G3507
VCC	5V
GND	GND
SDA	PA28
SCL	PA31

K230 camera:

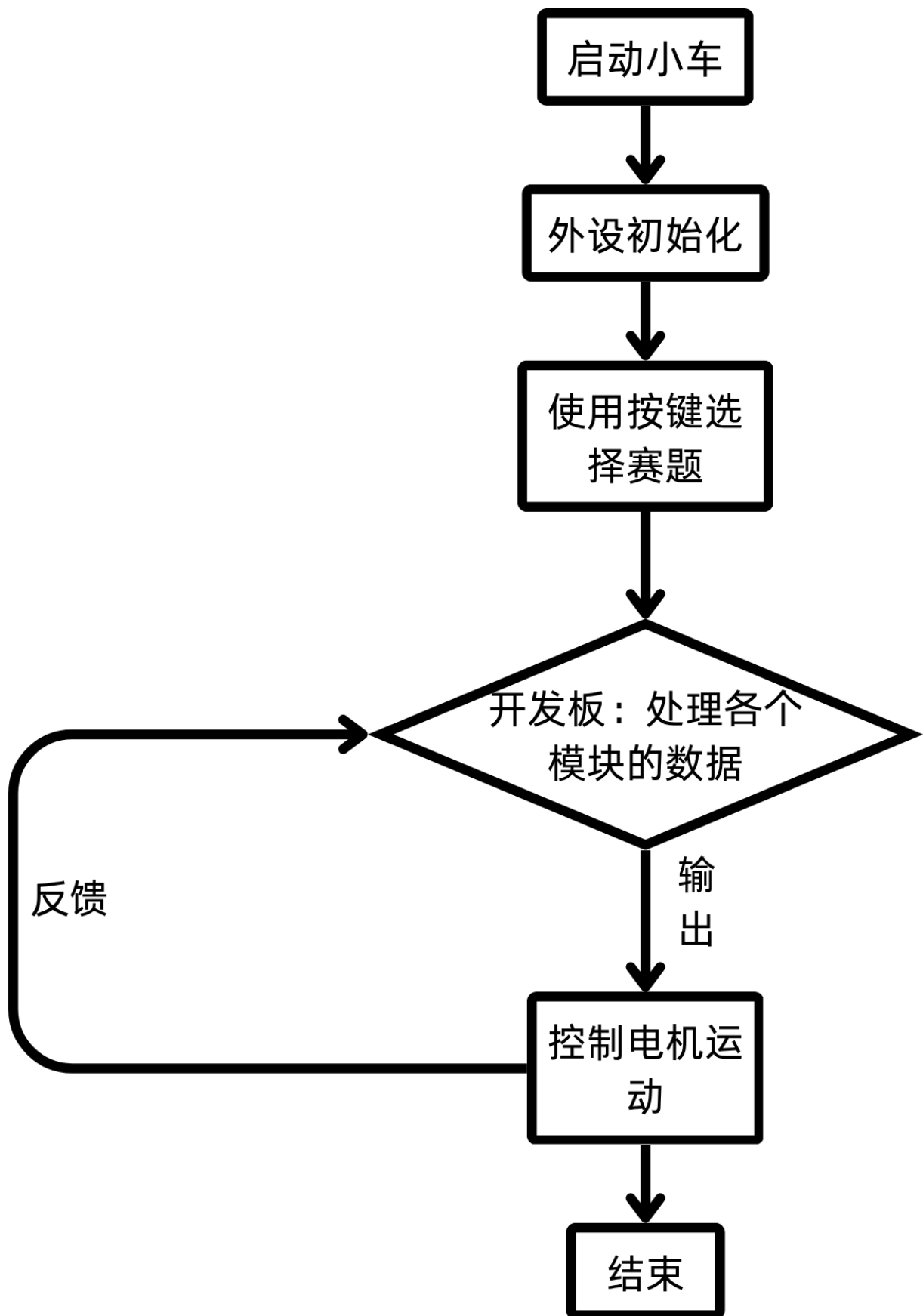
K230	MSPM0G3507
5V	5V
GND	GND
TXD	PA3
RXD	PA2

4.2 Physical Wiring Diagram



4.3 Control Principle

Car Tracking Control Flow:



1. 8-channel infrared sensors read the position of the black line -> calculate the deviation value
2. PID controller calculates the turning correction -> controls the differential steering of the motor
3. K230 camera detects the position of the steel ball -> servo motor PID controls the lever angle

4. Timer timeout or path point detection -> stop/change state

PID Control Parameter Description:

PID parameter	Fucntion	Adjust direction
P (Proportion)	The response speed is determined by the deviation; a larger deviation results in a larger output.	Too large: oscillating serpentine pattern; Too small: weak turning ability.
I (Integral)	Eliminate steady-state deviation and correct cumulative error.	Too big: overshoot; Too small: veers to one side and can't recover.
D (Differential)	Suppress oscillations and slow down in advance.	Too big: causing shaking; Too small: unable to stop.

5.Project Document Structure

```
Competition_PJ/
├─ main.c                # 主程序：外设初始化，主循环，状态机分发
├─ APP/
|   ├─ app_k230.c/h      # K230 串口协议 + PID 舵机控球（核心）
|   ├─ app_pid.c/h       # PID 控制器（位置式 + 增量式）
|   ├─ app_kalman.c/h    # 一维卡尔曼滤波器
|   └─ app_servo.c/h     # 舵机 PWM 驱动
├─ BSP/
|   ├─ Eight_Tracking/
|   |   └─ app_irtracking.c/h # 八路巡线 + 巡线PID + 组合任务(巡线+控球)
|   ├─ Motor/
|   |   ├─ app_motor.c/h    # 小车运动控制（Motion_Car_Control）
|   |   └─ app_motor_uart.c/h# 电机驱动板串口协议
|   ├─ OLED/
|   |   ├─ oled.c/h        # OLED 驱动 + 显示函数 OLED_time()
|   |   └─ oledfont.h      # 字体数据
|   ├─ Key/
|   |   └─ key.c/h         # 按键驱动 + 模式选择状态机
|   ├─ Questions/
|   |   └─ questions.c/h   # 竞赛任务状态机（Task 1~5）
|   ├─ Task/
|   |   └─ task.c/h        # 定时任务调度器（Scheduler_Run）
|   ├─ Timer/
|   |   └─ timer.c/h       # 1ms 系统时钟 Get_Time()
|   └─ uart.c/h           # 串口初始化（UART0 debug，UART1 电机，UART3 K230）
└─ ti_msp_dl_config.c/h  # TI 驱动库配置（SysConfig 生成）
```

6. Core Parameter Configuration

6.1 Servo Controller PID (app_k230.c)

```
#define PID_KP      0.08f  // Scale factor: Response strength, larger values
                        ••result in a stronger push
#define PID_KI      0.003f // Integral factor: Eliminates steady-state offset
#define PID_KD      2.86f  // Differential factor: Main damping, suppresses
                        oscillations

#define K230_CENTER  360.0f // k230 image center x pixels
#define K230_MIN      99.0f // Left boundary pixels
#define K230_MAX      599.0f // Right boundary pixels
#define FOV_CM        28.0f // Camera field of view width (cm)
#define PIPE_HALF_CM  12.5f // Half-length of water pipe (cm), full length of
                        water pipe 25cm

#define DEADBAND_PX   3.0f  // Dead zone (px), error <3px No action
#define RATE_LIMIT    6.0f  // Single-frame CC change limit, prevents sudden
                        servo rotation
#define KALMAN_Q       1.0f  // kalman process noise (high = fast response)
#define KALMAN_R       0.3f  // kalman measurement noise (high = smooth)
```

Two-way px_per_cm compensation (camera not perfectly centered):

```
Target >= 0 (right side): px_per_cm = (599-360)/12.5 = 19.12
Target < 0 (left side): px_per_cm = (360-99)/12.5 = 20.88
```

PID Tuning Guide:

Phenomenon	Which one to adjust?	Direction	Explanation
The ball wobbled back and forth near the center	↑ KD	Increase damping	The higher the KD value, the earlier the brakes are applied.
The ball stopped when it couldn't reach the target	↑ KP	Increase thrust	The larger the KP, the farther it can be pushed.
The ball bounced far after passing the target.	↓ KP	Reduce thrust	
After the ball stabilizes, it deviates to one side and fails to return to the center	↑ KI	Strengthen points	The higher the score, the faster the convergence.

Phenomenon	Which one to adjust?	Direction	Explanation
Servo vibration when the ball is stationary	↑ DEADBAND	Relax dead zone	

6.2 Tracking PID (app_irtracking.c)

```
#define IRTrack_Trun_KP    (150)    // Tracking P: Larger value means stronger
correction
#define IRTrack_Trun_KI    (0.15)   // Tracking I: Eliminates steady-state deviation
#define IRTrack_Trun_KD    (1.5)    // Tracking D: Suppresses oscillations
#define IRR_SPEED          250      // Default tracking speed (mm/s)
```

Line Inspection and Parameter Adjustment Guide:

Phenomenon	Which one to adjust?	Direction
Insufficient turning/crossing the line	↑ KP	Increase
Snake-like movement	↑ KD OR ↓ KP	Damping/Thrust Reduction
Always leaning to one side	↑ KI	Strengthen points

Inertia Compensation (for combined line-following and ball-handling tasks):

```
#define BALL_INERTIA_FWD    -0.8f    // Forward inertia compensation (cm), the larger
the value, the more the ball is pushed forward.
#define BALL_INERTIA_TURN  -0.8f    // Turning inertia compensation (cm)
#define BALL_START_COMP    -5.0f    // Start-up compensation (cm), pushes the ball
forward when the car accelerates and the ball lags
#define BALL_STOP_COMP      5.0f    // Stop compensation (cm), pulls the ball back
when the car stops and the ball is moving forward
#define RAMP_TIME_MS        3000    // Start-up acceleration time (ms), the larger the
value, the slower the start.
```

6.3 Line-following speed for each task (questions.c)

```
#define TASK1_SPEED        316      // Task 1: Pure Tracking Speed
#define TASK3_SPEED        130      // Task 3 A->B + Ball Control Speed
#define TASK4_SPEED        222      // Task 4: One lap + ball control speed
#define TASK5_SPEED        222      // Task 5: Velocity at a specified location
#define RAMP_TIME_MS        3000    // Start-up acceleration time
```

6.4 Task timeout settings (questions.c)

```
#define TASK1_MAX_MS 20000 // Task 1 Pure line tracking 20s Timeout
#define TASK2_MAX_MS 5000 // Task 2 Ball reciprocating 5s Timeout
#define TASK3_MAX_MS 8000 // Task 3 A->B + ball control 8s Timeout
#define TASK4_MAX_MS 30000 // Task 4 One lap + ball control 30s Timeout
#define TASK5_MAX_MS 30000 // Task 5 Designated position 30s Timeout
```

7. Task Description

7.1 State Machine Structure (questions.h)

Main_State:

Macro Instruction	value	Task Description
STOP_STATE	0	Stop
TASK_LINE_LAP	1	Task 1: Purely follow the line and return to point A in one lap, ≤20s
TASK_BALL_CTRL	2	Task 2: Steel ball O→+5→0→-5 (stop), <5s
TASK_AB_STAB	3	Task 3: A→B + steel ball stabilizes at O, ≤8s
TASK_LAP_STAB	4	Task 4: One rotation back to A + the steel ball stabilizes at O, ≤30s
TASK_LAP_POS	5	Task 5: One rotation back to A + steel ball to the designated position, ≤30s

Sub_State:

Macro Instruction	value	Task Description
SUB_IDLE	0	idle
SUB_INIT	1	Initialization (Set speed, start timing)
SUB_RUNNING	2	Running
SUB_DONE	3	Complete (Stop the car but maintain ball control; this state will also be entered if the timeout is exceeded)

7.2 Task Details

Task 1 (Pure Line Following):

- Line following only, no ball control
- Speed: TASK1_SPEED = 316 mm/s
- Stop at point A after passing 4 waypoints (A/B/C/D)
- Automatic stop after 20 seconds timeout

Task 2 (Ball Control):

- The car remains stationary, controlled only by the servo motor
- Procedure: 0 → +5cm (stop) → 0cm (stop) → -5cm (stop)
- Wait for the ball to approach the target (≤ 0.5 cm) for 30 consecutive frames before switching at each stage
- 5 seconds timeout
- Maintain the ball at -5cm position after the task is completed

Task 3 (A→B + Ball Control):

- Line following + servo ball control simultaneously
- Speed: TASK3_SPEED = 130 mm/s

From A Task 4 (One Loop + Ball Control):

- Line following + servo ball control simultaneously
- Speed: TASK4_SPEED = 222 mm/s
- One clockwise lap, passing 4 waypoints back to A
- Ball remains stable at center during travel
- Stop after 30 seconds, maintain ball control after stopping

Task 5 (Designated Position + Ball Control):

- Same as Task 4, but the target position of the ball can be specified (Target_Position)
- Speed: TASK5_SPEED = 222 mm/s
- Stop after 30 seconds, maintain ball control after stopping

Task 5 (Designated Position + Ball Control):

- Same as Task 4, but the target position of the ball can be specified (Target_Position)
- Speed: TASK5_SPEED = 222 mm/s
- Stop after 30 seconds, maintain ball control after stopping

8. Key Code Explanation

8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)

```
void k230_Servo_Control(float target_cm)
{
    // Parsing K230 serial port frames (UART3 interrupt for automatic data
    reception)
    int r; do { r = parse_one(g_buf, g_wr); } while (r > 0);
    if (!g_valid) return;

    // Coordinate transformation from cm to pixel (bidirectional px/cm compensation)
    float px_per_cm;
    if (target_cm >= 0.0f)
        px_per_cm = (K230_MAX - K230_CENTER) / PIPE_HALF_CM; // Right side
    else
        px_per_cm = (K230_CENTER - K230_MIN) / PIPE_HALF_CM; // Left side
    float target_px = K230_CENTER + target_cm * px_per_cm;

    // Dead time + positional PID + rate limiting + PWM output
    float error = target_px - Kalman_Filter(&kf_x, g_x);
    if (error > -DEADBAND_PX && error < DEADBAND_PX) return;
    float cc = PID_To_CC_Pos(&g_pid, x_flt);
    // ... Rate limiting + CC limiting...
    DL_TimerA_setCaptureCompareValue(PWM_Servo_INST, (uint16_t)cc,
    DL_TIMER_CC_0_INDEX);
}
```

PID formula (positional, app_pid.c): $u = K_p * e + K_i * \text{Integral}(e) + K_d * (e - e_{\text{last}})$

8.2 Line tracking — LineWalking() (app_irtracking.c)

```
void Linewalking(void)
{
    static int8_t err = 0;
    static u8 x1~x8;
    deal_IRdata(&x1,...,&x8); // I2C Reading 8-Channel Infrared

    if (x4 == 0 || x5 == 0) err = 0; // The black line in the middle, straight
    ahead.
    else if (x3 == 0) err = -1; // Left turn, right turn
    else if (x6 == 0) err = 1; // Right turn, left turn
    // ...more sensor combinations...

    pid_output_IRR = (int)(PID_IR_Calc(err));
    Motion_Car_Control(g_line_speed, 0, pid_output_IRR);
}
```

8.3 Line tracking + ball control combination — LineWalking_WithBall() (app_irtracking.c)

```
void Linewalking_withBall(float ball_target_cm)
{
    // Inertia compensation: Forward movement + Turning
    float compensated = ball_target_cm;
    compensated -= BALL_INERTIA_FWD;
    if (pid_output_IRR > 0)
        compensated -= BALL_INERTIA_TURN * pid_output_IRR / 5000.0f;

    // Slow acceleration from start: The initial RAMP_TIME_MS speed gradually
    // increases from 0 milliseconds.
    uint32_t et = Get_Time() - start_time;
    if (et < RAMP_TIME_MS) {
        float ramp = (float)et / RAMP_TIME_MS;
        g_line_speed = (int)(g_line_speed * ramp);
    }

    Linewalking();                // Line patrol
    K230_Receive_Process();       // Receive K230
    K230_Servo_Control(compensated); // Ball control
}
```

8.4 Task 2: Steel ball reciprocating (questions.c)

```
// Three stages: +5cm (1.8s) -> 0cm (1.8s) -> -5cm (1.8s) -> DONE (hold at -5)
case SUB_RUNNING: {
    static int phase = 0;
    static uint32_t t0 = 0;
    if (t0 == 0) t0 = Get_Time();

    float target_cm;
    if (phase == 0) target_cm = +5.0f;
    else if (phase == 1) target_cm = 0.0f;
    else target_cm = -5.0f;

    K230_Receive_Process();
    K230_Servo_Control(target_cm);

    if (Get_Time() - t0 >= 1800) { // 1.8 seconds per phase
        t0 = Get_Time();
        if (phase == 0) phase = 1;
        else if (phase == 1) phase = 2;
        else { phase = 0; Sub_State = SUB_DONE; }
    }
    break;
}

case SUB_DONE:
    K230_Receive_Process();
```

```
K230_Servo_Control(-5.0f);    // Keep the ball at -5cm position  
break;
```

9. Operation Procedure

1. Power on after wiring. The OLED displays "K230 Init..." Wait 4 seconds.
2. Enter the mode selection interface. The OLED displays "Mode:1".
3. **Long press K1** to switch modes (1~5).
4. **Short press K1** to confirm entering the current mode.
5. The car starts performing the task. The OLED displays the task name + timer.
6. **Short press K1 during task execution:** Pause; **Long press K1:** Exit and return to mode selection.
7. After the task is completed/timeout, the car stops, but the **ball control task continues** (the ball remains in the target position).
8. Long press K1 to exit and return to mode selection.